

Web 前端开发基础

| 从 CampusHub 页面到可交互的 Vue 应用

HTML · CSS · JavaScript · Vue · API

本讲学习目标

完成本讲后，你应该能够：

1. 说清楚 **HTML**、**CSS**、**JavaScript** 各自负责什么；
2. 读懂一个简单页面的 DOM 结构和 CSS 布局；
3. 使用 JavaScript 处理事件、更新页面、请求后端数据；
4. 使用 Vue 编写包含列表、表单和状态的组件；
5. 在 AI 帮助下开发页面，并能审查生成结果。

目标不是记住所有语法，而是建立一个可以继续学习的前端心智模型。

今天要完成的页面

CampusHub · 互助需求列表

快递代取

帮忙取一个快递

地点：南园驿站 时间：18:00 前

学习辅导

寻找 Java 学习搭子

地点：图书馆 时间：周五晚

我们将逐步实现：展示 → 美化 → 交互 → 获取数据 → 组件化。

先建立一个最简单的模型

HTML

页面上**有什么**

标题、列表、表单、按钮

CSS

页面**长什么样**

颜色、间距、位置、响应式

JavaScript

页面**如何变化**

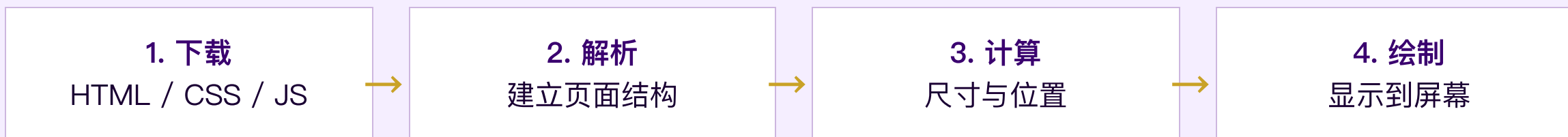
点击、输入、请求、更新

HTML 是骨架，CSS 是外观，JavaScript 是行为。

PART 1

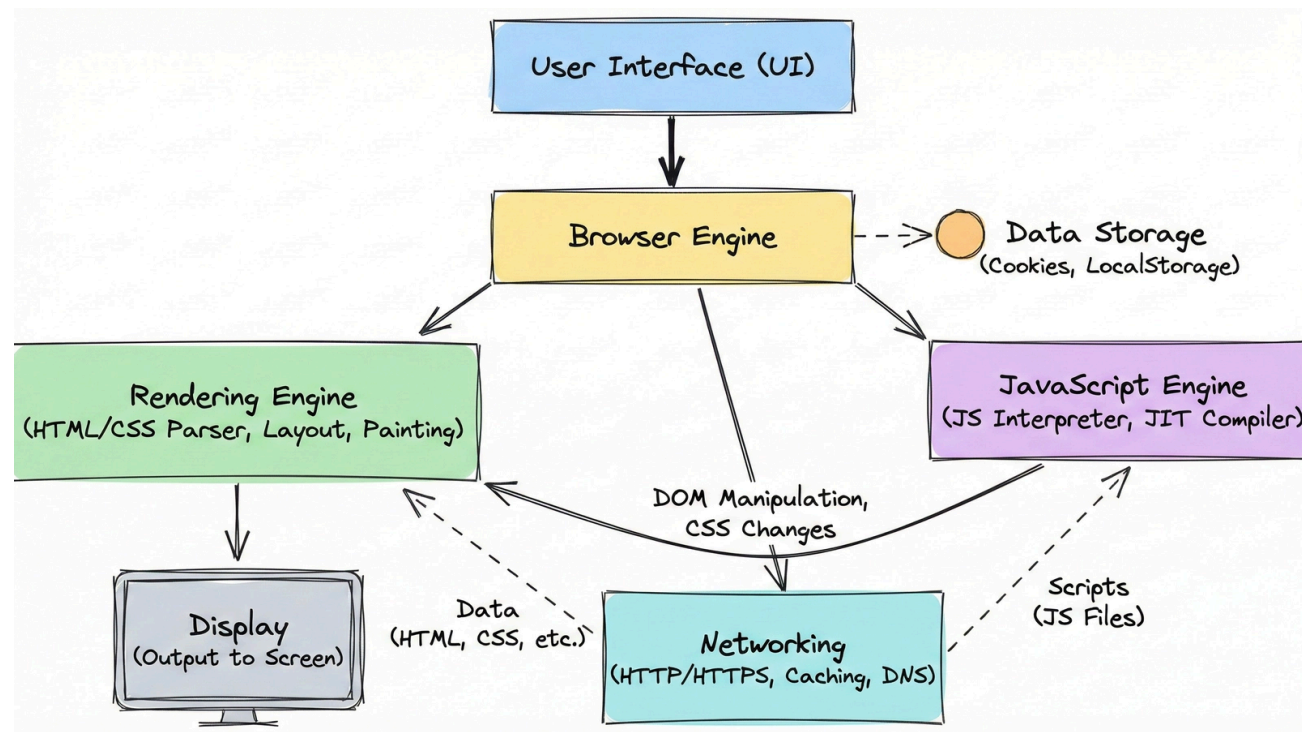
浏览器如何显示一个页面？

从文件到屏幕：只看四步



浏览器不是“打开一张图片”，而是读取代码、建立结构、计算布局，再持续响应用户操作。

浏览器内部：谁在完成这些工作？



来源：DEV Community, “How a Browser Works: A Beginner-Friendly Guide”

观察图中的两条主线：网络模块取得资源；渲染引擎与 JavaScript 引擎共同更新屏幕。

HTML：描述页面内容与结构

```
<article class="demand-card">
  <span class="category">快递代取</span>
  <h2>帮忙取一个快递</h2>
  <p>南园驿站, 18:00 前</p>
  <button>我要接单</button>
</article>
```

读代码时先问三个问题：

- 哪个元素是整体？—— `article`
- 哪些是内容？—— `span`、`h2`、`p`
- 哪个元素可以操作？—— `button`

HTML 会变成一棵 DOM 树

```
article.demand-card
├── span.category    "快递代取"
├── h2               "帮忙取一个快递"
├── p                "南园驿站, 18:00 前"
└── button           "我要接单"
```

DOM (Document Object Model): 浏览器在内存中建立的页面对象树。

JavaScript 修改页面, 本质上是在修改这棵树中的节点。

常用 HTML：先掌握这 8 个

类型	标签	用途
页面区域	<code>header</code> <code>main</code> <code>nav</code>	组织页面的大结构
内容	<code>h1 ~ h3</code> <code>p</code>	标题和文本
列表	<code>ul</code> <code>li</code>	多条同类内容
操作	<code>button</code> <code>a</code>	命令和页面跳转
输入	<code>input</code> <code>select</code> <code>textarea</code>	收集用户数据
容器	<code>div</code> <code>section</code> <code>article</code>	组合相关内容

先写有意义的结构，再考虑颜色和位置。

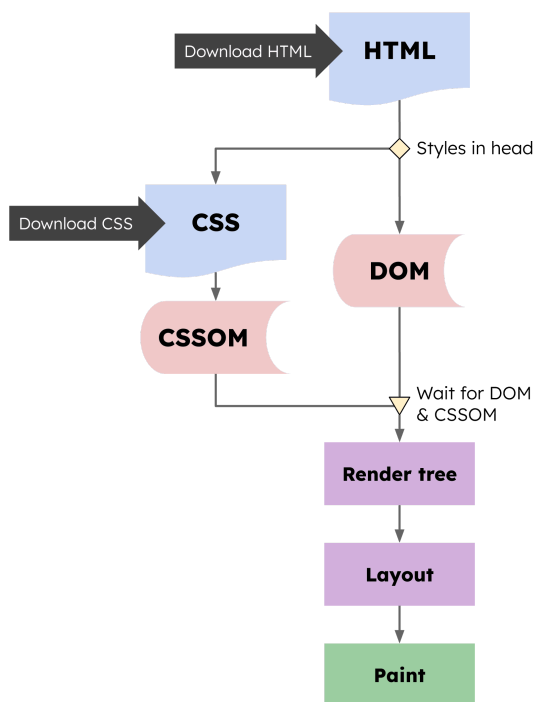
CSS：选择元素，再设置样式

```
.demand-card {  
  padding: 16px;  
  border: 1px solid #ddd;  
  border-radius: 6px;  
}  
  
.demand-card button {  
  background: #4d0099;  
  color: white;  
}
```

选择器		声明
<code>.demand-card</code>	→	<code>padding: 16px</code>

CSS 的核心就是：找到谁，设置什么。

浏览器渲染流水线



读图顺序

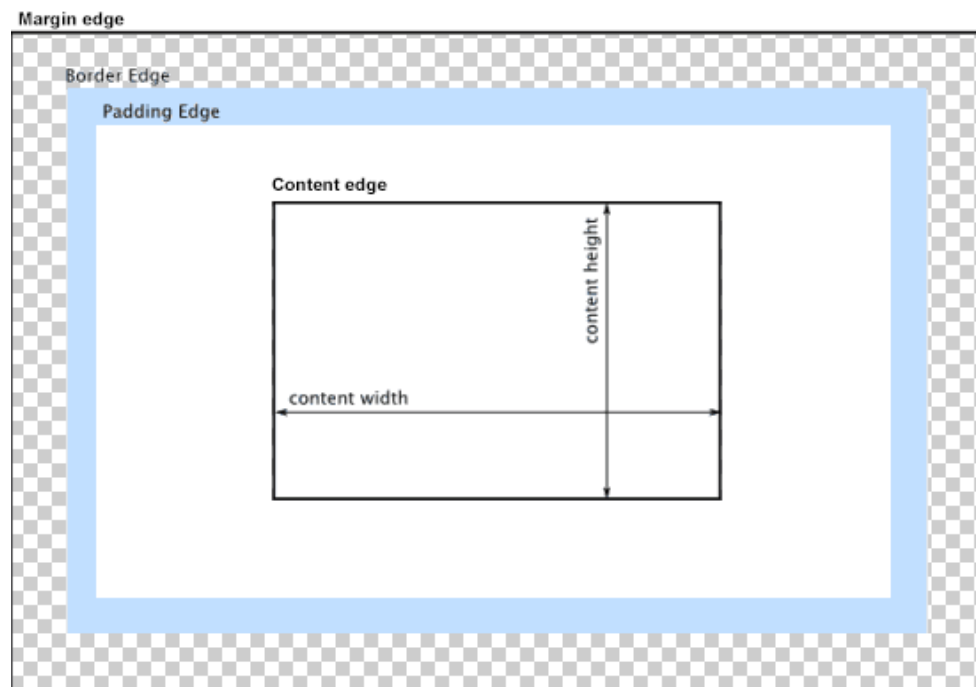
1. HTML 形成 DOM;
2. CSS 形成 CSSOM;
3. 二者合成 Render Tree;
4. Layout 计算位置, Paint 生成像素。

只有可见节点进入渲染树。

- 修改颜色通常触发**重绘 (Repaint)**;
- 修改尺寸或位置可能触发**重新布局 (Reflow)**;

来源: Critical Rendering Path 图解, [pozafly.github.io](https://github.com/pozafly)

盒模型：每个元素都是一个盒子



从内向外读

Content: 实际内容

Padding: 内容与边框的距离

Border: 盒子的可见边界

Margin: 与其他盒子的距离

来源：MDN Web Docs, CSS Box Model

页面“挤在一起”时，通常先检查：

- `width` / `height`
- `padding`
- `border`
- `margin`

Flex: 把一组元素排成一行或一列

```
.demand-list {  
  display: flex;  
  gap: 16px;  
  flex-wrap: wrap;  
}  
  
.demand-card {  
  flex: 1 1 280px;  
}
```

需求卡片 A



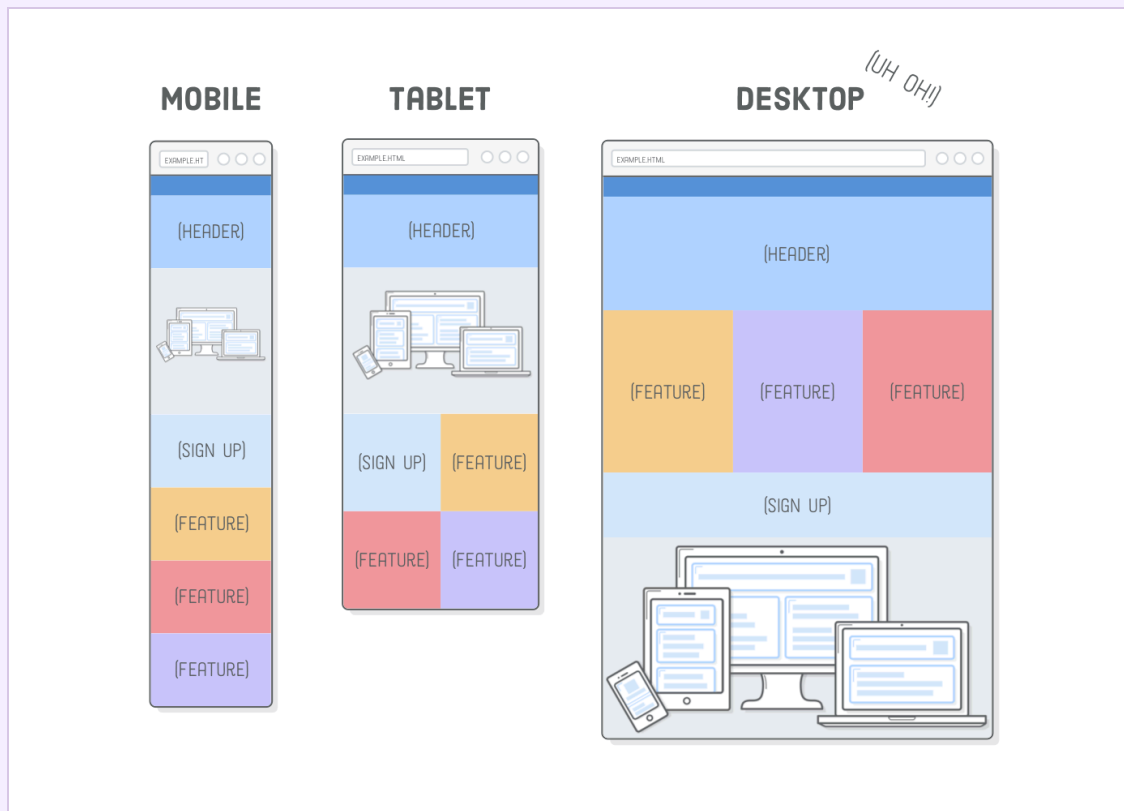
需求卡片 B



需求卡片 C

`gap` 控制间距, `flex-wrap` 允许空间不足时换行。

响应式：同一页面适应不同宽度



观察同一组内容

手机：空间窄，卡片排成一列。

平板：空间增加，部分内容排成两列。

桌面：空间充足，三个功能区并排。

响应式不是把桌面页面整体缩小，而是让内容根据可用宽度重新排布。

来源：HTML & CSS Is Hard, Responsive Images Tutorial

```
.demand-list { display: grid; grid-template-columns: repeat(3, 1fr); }  
@media (max-width: 600px) { .demand-list { grid-template-columns: 1fr; } }
```

JavaScript: 响应事件并改变页面

```
<button id="acceptBtn">我要接单</button>  
<p id="message"></p>
```

```
const button = document.querySelector('#acceptBtn')  
const message = document.querySelector('#message')  
  
button.addEventListener('click', () => {  
  message.textContent = '接单成功'  
})
```

用户点击 → 事件触发 → 执行函数 → DOM 更新

JavaScript 中最常见的四类数据

```
const title = '帮忙取快递'      // 字符串
const reward = 5                 // 数字
const accepted = false          // 布尔值

const demand = {                // 对象
  id: 101,
  title: '帮忙取快递',
  category: 'express'
}

const demands = [demand]        // 数组
```

前端页面通常就是把对象和数组中的数据显示出来。

JavaScript 的四类核心能力

操作页面

查询、创建、修改 DOM 节点

`querySelector` · `textContent`

响应用户

处理点击、输入、提交等事件

`addEventListener`

管理数据

计算、筛选并维护页面状态

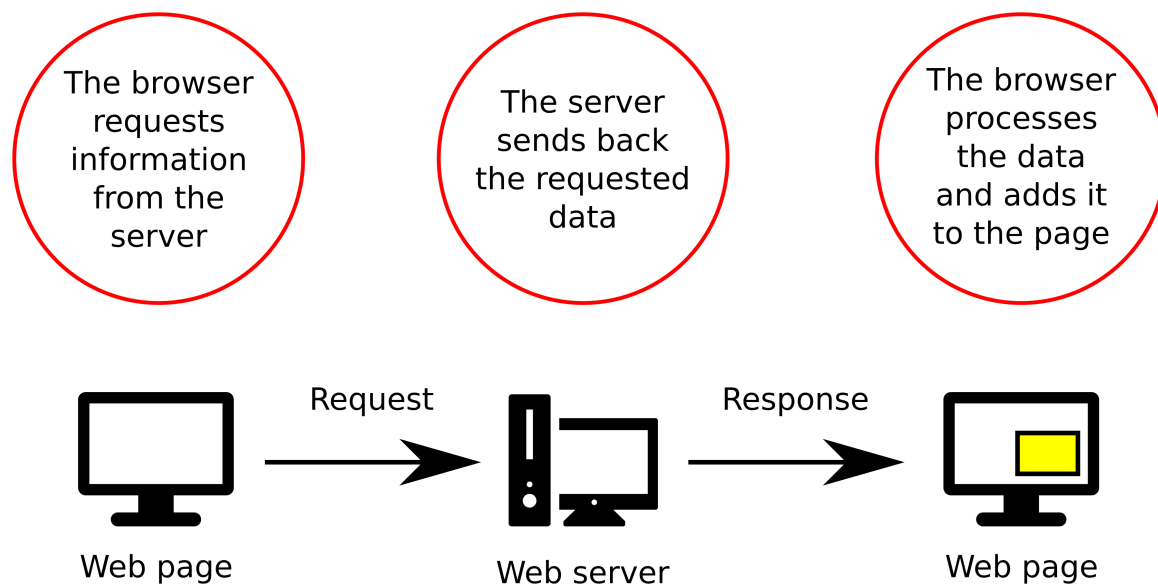
`map` · `filter`

访问服务

异步调用后端 API

`fetch` · `async/await`

AJAX: 页面为何不再整页刷新?



来源: Introduction to Web Mapping, "How AJAX works"

带来的能力

更快的反馈、更少的数据传输、接近桌面应用的连续体验

引入的复杂度

异步状态、错误恢复、历史导航、前后端接口一致性

今天通常使用 `fetch` 完成 AJAX。技术名称变了，核心思想仍是：后台取数据，局部更新页面。

从后端获取数据：Fetch API

```
async function loadDemands() {  
  const response = await fetch('/api/demands')  
  
  if (!response.ok) {  
    throw new Error('加载失败')  
  }  
  
  const demands = await response.json()  
  return demands  
}
```

浏览器
`fetch`



HTTP 请求
`GET /api/demands`



后端响应
JSON 数据

异步不是“同时完成”

1. 页面先显示“正在加载...”
2. 浏览器发送请求
3. 用户仍然可以操作页面
4. 后端返回数据
5. 页面更新为需求列表

前端必须考虑至少三种状态：

Loading

正在加载

Success

显示数据

Error

失败与重试

PART 2

为什么需要 Vue?

原生 DOM 操作的问题

```
const list = document.querySelector('#demandList')

demands.forEach(demand => {
  const item = document.createElement('li')
  item.textContent = demand.title
  list.appendChild(item)
})
```

当页面状态增加时，我们需要手工维护：

- 数据变了，哪些节点要更新？
- 节点应该新增、删除还是复用？
- 多个事件修改同一数据时怎么办？

Vue 帮助我们注意力从“操作 DOM”转向“描述页面”。

Vue 的核心：数据驱动视图

数据 state
↓
模板 template
↓
页面 view
↑
用户事件 event

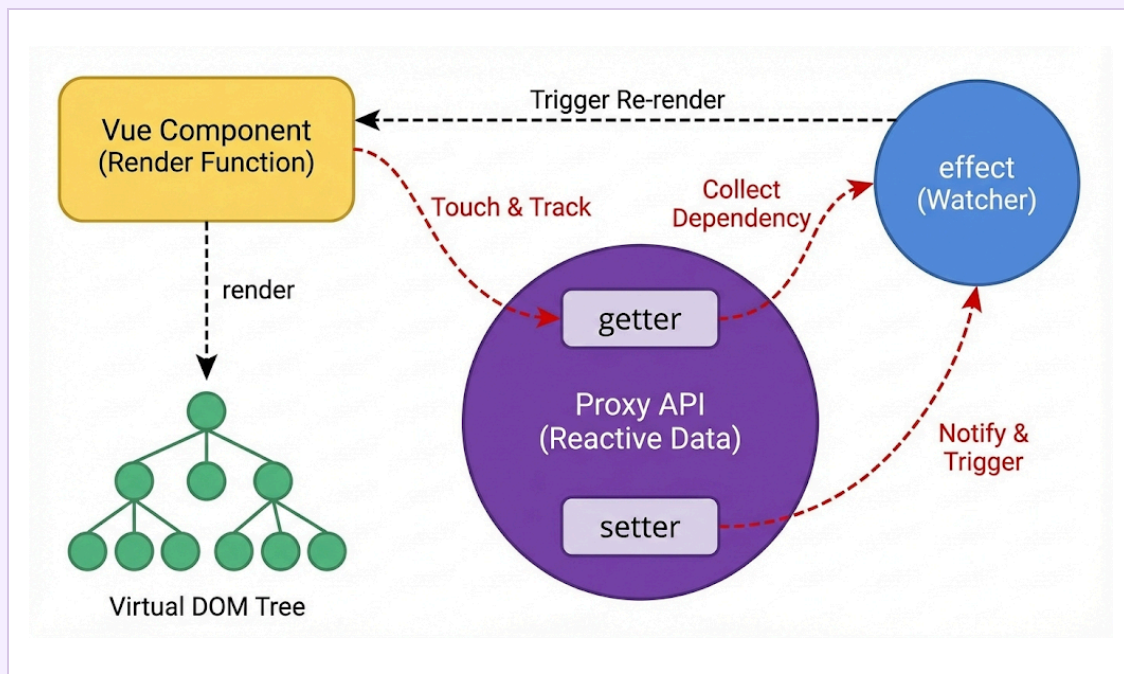
```
<script setup>
import { ref } from 'vue'

const count = ref(0)
</script>

<template>
  <button @click="count++">接单 {{ count }} 次</button>
</template>
```

我们只修改 `count`，Vue 负责更新页面。

Vue 响应式：数据与界面形成闭环



图中的两个关键动作

Track：渲染时记录组件读取了哪些数据。

Trigger：数据变化时，只通知依赖它的组件更新。

`ref()` 和 `reactive()` 用来创建响应式状态。

响应式负责同步视图，业务状态仍需开发者设计。

来源：Tangyuxian, Vue 3 响应式原理图

一个 Vue 文件包含三个部分

```
<script setup>
const demand = {
  title: '帮忙取一个快递',
  location: '南园驿站'
}
</script>

<template>
  <article class="card">
    <h2>{{ demand.title }}</h2>
    <p>{{ demand.location }}</p>
  </article>
</template>

<style scoped>
.card { padding: 16px; border: 1px solid #ddd; }
</style>
```

Vite: 把 Vue 项目组织并运行起来

```
campushub-web/  
├─ index.html      浏览器入口  
├─ package.json    依赖与运行命令  
└─ src/  
    ├─ main.js      创建并挂载 Vue 应用  
    ├─ App.vue       根组件  
    ├─ components/  可复用组件  
    └─ views/        页面级组件
```

```
npm create vite@latest campushub-web -- --template vue  
cd campushub-web  
npm install  
npm run dev
```

Node.js 在这里首先是前端工具链的运行环境：Vite、依赖安装和构建依赖它；页面中的 JavaScript 最终仍在浏览器执行。

Vue 模板：记住五种常用写法

写法	含义	示例
<code>{{ value }}</code>	显示数据	<code>{{ demand.title }}</code>
<code>:属性="值"</code>	动态属性	<code>:disabled="loading"</code>
<code>@事件="处理"</code>	监听事件	<code>@click="accept"</code>
<code>v-if="条件"</code>	条件显示	<code>v-if="error"</code>
<code>v-for="项 in 列表"</code>	循环显示	<code>v-for="d in demands"</code>

先会读，再会写；先解决真实页面问题，再记指令名称。

案例：显示 CampusHub 需求列表

```
<script setup>
const demands = [
  { id: 1, title: '帮忙取快递', category: '快递代取' },
  { id: 2, title: 'Java 学习搭子', category: '学习辅导' }
]
</script>

<template>
  <article v-for="demand in demands" :key="demand.id">
    <span>{{ demand.category }}</span>
    <h2>{{ demand.title }}</h2>
    <button>查看详情</button>
  </article>
</template>
```

`:key` 帮助 Vue 稳定识别每一条数据。

表单：让用户发布需求

```
<script setup>
import { reactive } from 'vue'

const form = reactive({
  title: '',
  category: '快递代取'
})

function submit() {
  console.log(form)
}
</script>

<template>
  <form @submit.prevent="submit">
    <input v-model="form.title" placeholder="需求标题">
    <select v-model="form.category">
      <option>快递代取</option>
      <option>学习辅导</option>
    </select>
    <button>发布</button>
  </form>
</template>
```

组件：把页面拆成可理解的部分

```
DemandPage
├─ FilterBar      筛选条件
├─ DemandList     需求列表
│   └─ DemandCard 单条需求
└─ LoadingState   加载反馈
```

拆分组件时问三个问题：

1. 它是否有清晰、独立的职责？
2. 它是否会在多处重复使用？
3. 拆分后接口是否比原代码更容易理解？

不要为了“组件化”把每个标签都拆成组件。

组件协作：父传数据，子发事件

```
<!-- DemandCard.vue -->
<script setup>
defineProps({ demand: Object })
const emit = defineEmits(['accept'])
</script>

<template>
  <article>
    <h2>{{ demand.title }}</h2>
    <button @click="emit('accept', demand.id)">接单</button>
  </article>
</template>
```

父组件 -- props 数据 --> 子组件

父组件 <-- emit 事件 --- 子组件

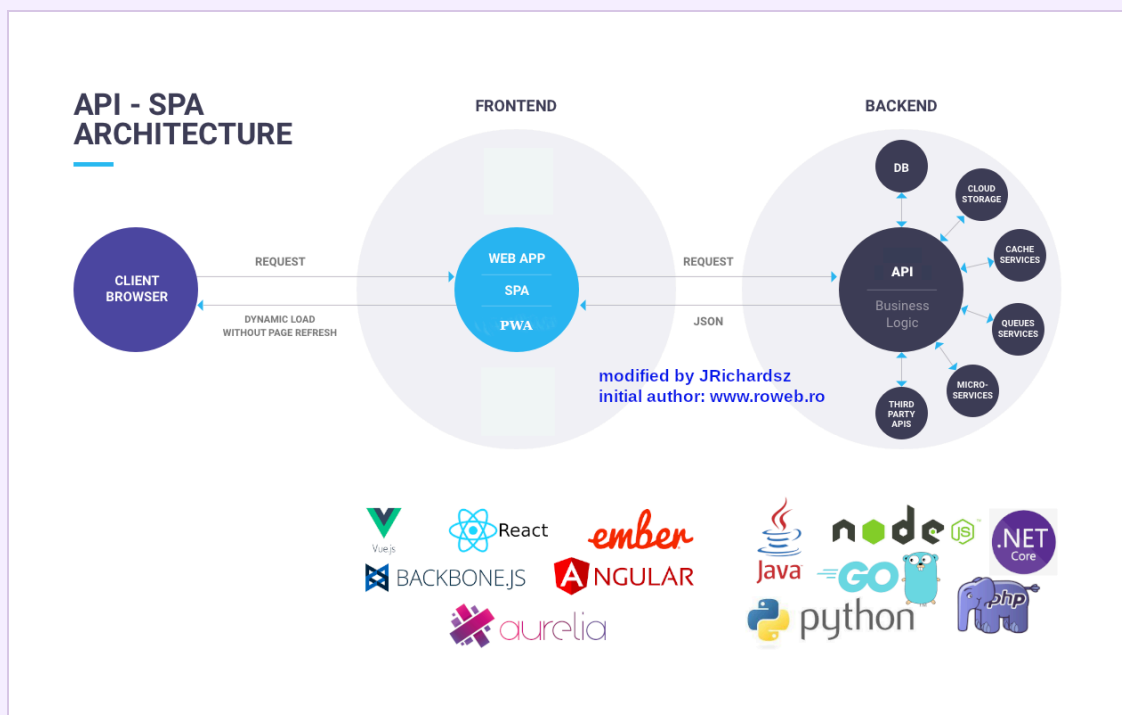
Vue 调用后端 API

```
<script setup>
import { onMounted, ref } from 'vue'

const demands = ref([])
const loading = ref(true)
const error = ref('')

onMounted(async () => {
  try {
    const response = await fetch('/api/demands')
    if (!response.ok) throw new Error('请求失败')
    demands.value = await response.json()
  } catch (e) {
    error.value = e.message
  } finally {
    loading.value = false
  }
})
</script>
```

前端和后端如何分工？



前端

展示、交互、即时提示、调用 API、处理加载与失败状态。

后端

业务规则、权威校验、认证授权、数据持久化、提供 API。

来源：JRichardsz, API-SPA Architecture

前端隐藏按钮不等于完成权限控制。用户可以绕过页面直接调用 API，真正的权限判断必须由后端完成。

一个完整交互的时序

用户点击“接单”

↓

Vue 禁用按钮，显示“处理中”

↓

POST /api/orders

↓

后端检查登录、权限和订单状态

↓

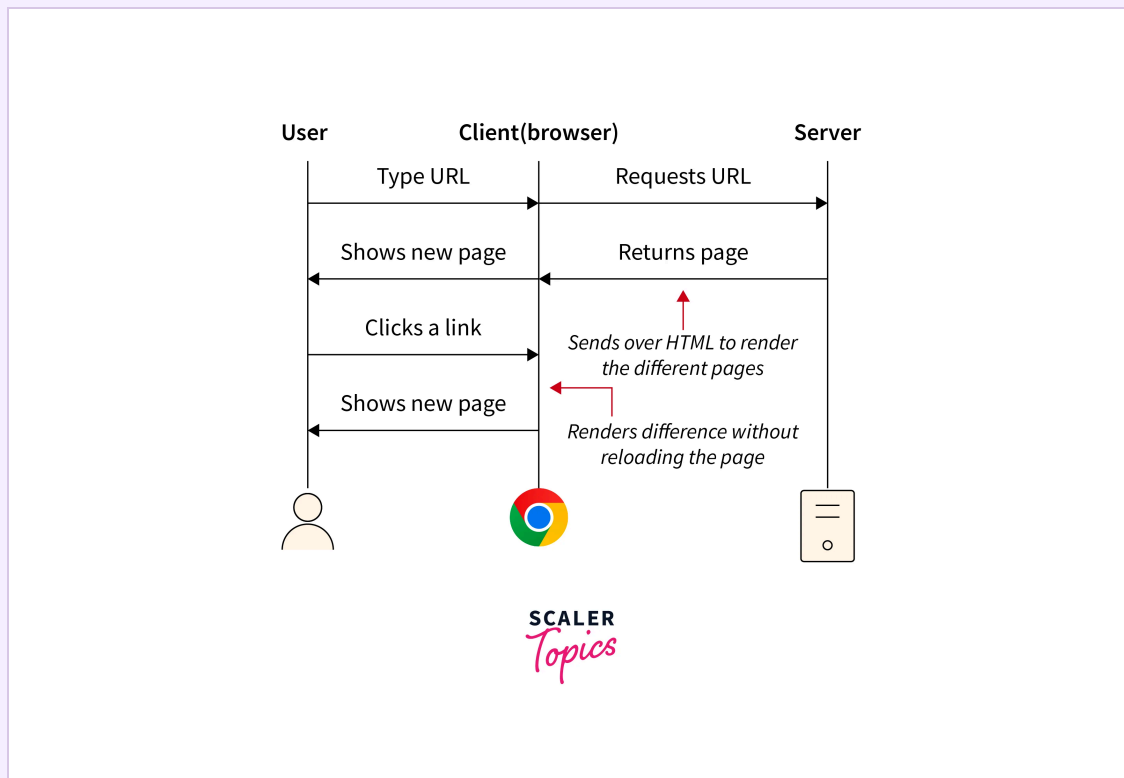
成功：更新页面状态

失败：显示可理解的错误信息

前端体验与后端正确性必须同时成立。

客户端路由与服务端路由：不是二选一

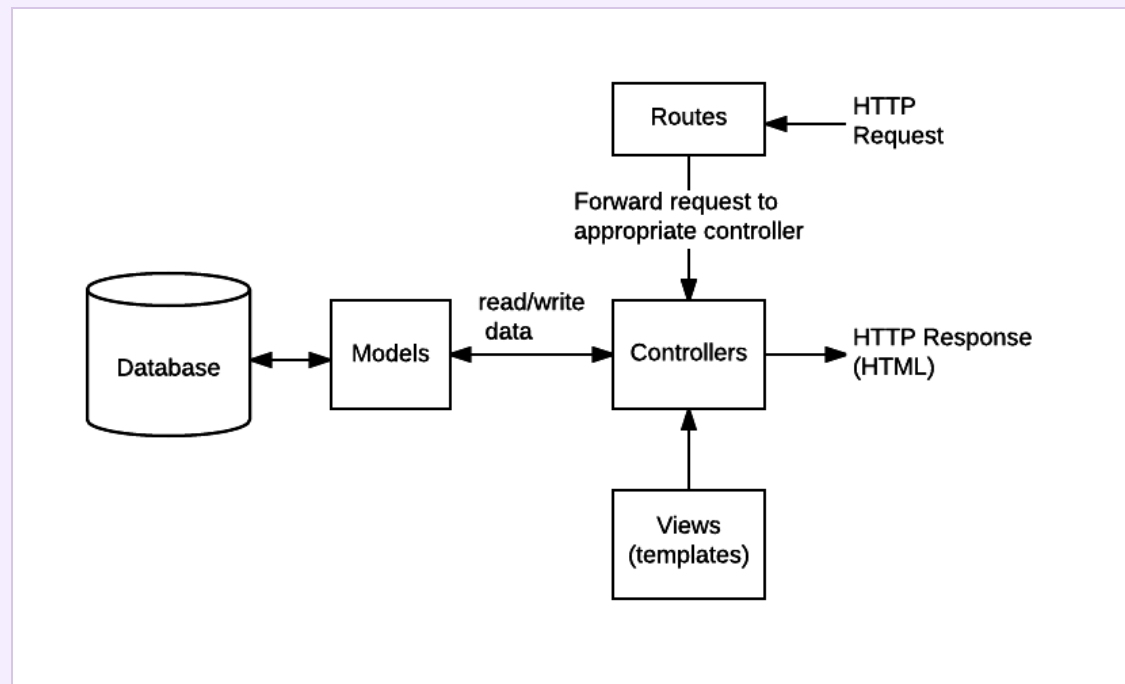
客户端路由：URL → 页面组件



`/demands/101` → Vue Router → `DemandDetail.vue`

发生在浏览器内，切换组件时不重新加载页面框架。

服务端 API 路由：请求 → 处理函数



`GET /api/demands/101` → Handler → JSON

发生在服务器内，执行查询、校验和业务逻辑。

左：Scaler Topics, Client-side Routing；右：MDN Express Routes and Controllers

CampusHub 中二者同时工作：客户端路由决定“显示什么”，服务端路由决定“执行什么并返回什么”。

PART 3

AI 协作开发前端页面

不要只说“帮我写一个页面”

含糊任务

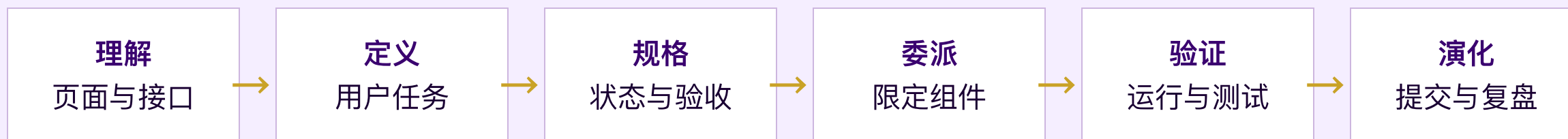
帮我写一个 CampusHub 需求列表页面。

可执行任务

使用 Vue 3 + `<script setup>` 实现 `DemandList.vue`。
输入数据字段: `id`、`title`、`category`、`location`、`status`。
要求: 支持分类筛选; 包含 `loading`、`empty`、`error` 三种状态;
点击“接单”只触发 `accept(id)` 事件, 不直接修改订单状态。
不要引入新的 UI 库。请同时给出组件测试要点。

差别不只是“提示词更长”, 而是**问题定义更完整**。

前端小任务的 AI 协作六步法



关键约束

- 指定允许修改的文件；
- 提供 API 契约和示例数据；
- 明确 Loading、Empty、Error、Success 状态；
- 用浏览器和测试验证，而不是相信 AI 的描述。

审查 AI 生成的前端代码

检查项	典型问题
功能	只覆盖正常流程，遗漏空数据和失败状态
数据	字段名与后端 API 不一致
状态	请求完成后忘记关闭 <code>loading</code>
组件	组件过大，或拆分得过于零碎
安全	使用 <code>v-html</code> 直接渲染用户输入
可维护性	大量重复代码、魔法字符串、职责混乱
可用性	按钮无反馈、错误信息无法理解

AI 能快速生成初稿，软件工程知识决定初稿能否进入项目。

小练习 1：读懂并修改静态页面

给定一个 CampusHub 需求卡片：

1. 用语义化 HTML 调整页面结构；
2. 使用 Flex 完成卡片布局；
3. 在窄屏下改为单列；
4. 增加“接单成功”的点击反馈；
5. 解释 HTML、CSS、JavaScript 分别修改了什么。

提交： 页面截图 + 三层职责说明 + Git diff。

小练习 2: Vue 需求列表

实现 `DemandList.vue` :

- 使用 `v-for` 显示需求;
- 使用 `v-model` 绑定分类筛选;
- 使用 `v-if` 处理空列表;
- 点击卡片触发 `view-detail` 事件;
- 从模拟 API 加载数据;
- 显示 Loading、Error、Success 状态。

AI 可以生成初稿, 但必须提交:

- 任务 Spec;
- 人工发现并修正的问题;
- 最终验证结果。

本讲知识地图

浏览器

- └─ HTML: 内容与结构 → DOM 树
- └─ CSS: 样式与布局 → 渲染树 / 盒模型 / Flex / 响应式
- └─ JavaScript: DOM / 事件 / 数据 / AJAX → Fetch / 异步状态

Vue

- └─ MVVM 与响应式: 数据驱动视图
- └─ 模板与常用指令
- └─ 组件 / Props / Emits
- └─ Vite 项目结构
- └─ API 请求 / 两类路由 / 页面状态

AI 协作

- └─ 定义问题 → 编写规格 → 委派实现 → 审查验证

最重要的四个结论

1. 前端不是“把页面画出来”，而是管理结构、状态和用户交互；
2. HTML / CSS / JavaScript 各司其职，分离关注点使代码更容易维护；
3. Vue 用数据驱动视图，组件是组织复杂页面的基本单元；
4. AI 可以提高实现速度，但页面状态、接口契约和质量必须由工程师验证。

课后任务

必做

完成 CampusHub `DemandList.vue` 小练习，并提交：

1. 可运行代码；
2. 页面四种状态截图：Loading / Empty / Error / Success；
3. AI 协作记录：任务 Spec、关键输出、人工修改；
4. 200 字反思：AI 遗漏了什么？你如何发现？

选做

- 增加移动端响应式布局；
- 为筛选和错误状态编写组件测试。

Web 前端开发基础

| 从“页面能显示”走向“组件可维护、结果可验证”

下一讲：Web 后端开发基础